

NLP 小作业 1

基于卷积神经网络的文本情感分类

智科 221900180 田永铭 tianym2022@smail.nju.edu.cn

2025 年 4 月 8 日

目录

一、任务简述	1
二、方法介绍和实现 (包括模块介绍和超参数设置)	2
2.1 数据集介绍	2
2.2 Glove 预训练词向量	3
2.3 实现卷积神经网络类	4
2.4 实现卷积神经网络的训练	5
三、实验	6
3.1 五分类实验	6
3.2 消融实验-超参数变化	7
3.3 拓展实验到二分类任务上	7
3.4 与往年参考准确率对比	9
3.5 案例分析	9
四、总结与反思	10

一、任务简述

该实验旨在利用卷积神经网络,在电影评论数据集 (Rotten Tomatoes) 上实现文本情感分类。我主要参考论文“[Convolutional Neural Networks for Sentence Classification](#)”[1], 采用网络上的[预处理后的电影评论数据集](#)(提取码: otz1), 利用[GloVe 预训练的词向量](#)[2], 编

写卷积神经网络方法，训练并且最终在测试集上达到了 **46.47%**(五分类) 和 **89.53%** (二分类) 的精度。演示视频链接为[田永铭 221900180 NLP 小作业 1 视频演示](#)。注意视频中不包含案例分析部分，这是老师提出新要求后我新补充的内容，在本报告中有体现，见 3.5 节。

二、方法介绍和实现 (包括模块介绍和超参数设置)

2.1 数据集介绍

Rotten Tomatoes 数据集是一项广泛用于自然语言处理 (NLP) 领域的情感分类任务数据集。它包含从[Rotten Tomatoes](#) 网站上收集的电影评论。经过预处理后，每条数据由一些单词集合 + 情感标签 (0, 1, 2, 3, 4 五个整数，数字越大评论越正面) 组成。该数据集包含 8545 条训练数据，1102 条验证数据和 2211 条测试数据。

数据示例如下：

评论: "['you', "'ll'", 'probably', 'love', 'it']"

标签: 4

我编写了一个数据集类来加载这个数据集 (其中 tokens2id 是数据集将 token 映射为 id 的预处理 txt 文件)，相关代码如下：

```
1 # 数据集类
2 class SentimentDataset(Dataset):
3     def __init__(self, file_path, token2id_path, max_len=MAX_LEN):
4         self.data = pd.read_csv(file_path)
5         self.token2id = pd.read_csv(token2id_path, header=None, index_col=0)[1].
            to_dict()
6         self.max_len = max_len
7     def __len__(self):
8         return len(self.data)
9
10    def __getitem__(self, idx):
11        sentence = eval(self.data.iloc[idx, 0])
12        label = self.data.iloc[idx, 1]
13        encoded = [self.token2id.get(token, 0) for token in sentence]
14        if len(encoded) < self.max_len:
15            encoded.extend([0] * (self.max_len - len(encoded)))
16        else:
17            encoded = encoded[:self.max_len]
18        return torch.tensor(encoded, dtype=torch.long), torch.tensor(label,
            dtype=torch.long)
```

2.2 GloVe 预训练词向量

GloVe (Global Vectors for Word Representation) 是由斯坦福大学提出的一种用于生成词向量的模型 [2]。它基于词共现矩阵进行训练，以捕捉词语之间的语义关系。与基于局部上下文窗口的 Word2Vec 不同，GloVe 利用全局统计信息，使词向量在语义空间中具有更好的表现。

GloVe 的核心思想是通过词共现概率矩阵来学习词向量。具体而言，假设有一个词汇表 V ，词汇 i 和 j 的共现次数为 X_{ij} ，则共现概率可以定义为：

$$P(i|j) = \frac{X_{ij}}{\sum_k X_{kj}}$$

GloVe 的目标是找到合适的词向量，使得以下关系成立：

$$F(w_i, w_j, b_i, b_j) = \log(X_{ij})$$

其中 w_i 和 w_j 分别为词汇 i 和 j 的词向量， b_i 和 b_j 为对应的偏置项， F 是目标函数，用于最小化损失。

GloVe 使用加权平方误差作为目标函数，其形式为：

$$J = \sum_{i,j=1}^V f(X_{ij}) (w_i^T w_j + b_i + b_j - \log(X_{ij}))^2$$

其中： $f(X_{ij})$ 是加权函数，用于降低稀疏数据对训练的影响。我们一般选择 $f(x) = \left(\frac{x}{x_{\max}}\right)^\alpha$ 。

GloVe 的训练过程包括以下步骤：

1. 构建词共现矩阵：扫描文本语料，统计词对共现次数。
2. 初始化词向量和偏置项。
3. 使用梯度下降法优化目标函数。
4. 生成最终的词向量。

在本实验中，我选择采用的 GloVe 词向量的维度是 300 维。相关加载 GloVe 词向量的代码如下：

```
1 # 加载 GloVe 词向量
2 def load_glove_embeddings(glove_path, token2id, embed_dim=EMBED_DIM):
3     embeddings = np.random.uniform(-0.25, 0.25, (len(token2id), embed_dim))
4     with open(glove_path, 'r', encoding='utf-8') as f:
5         for line in f:
6             values = line.strip().split()
```

```
7     word = values[0]
8     vector = np.asarray(values[1:], dtype=np.float32)
9     if word in token2id:
10         embeddings[token2id[word]] = vector
11     return torch.tensor(embeddings, dtype=torch.float)
```

2.3 实现卷积神经网络类

参考论文“[Convolutional Neural Networks for Sentence Classification](#)”[1], 我设计了一个卷积神经网络, 其作为机器学习基础知识这里不多介绍。该网络总体结构是“嵌入层 + 卷积层 + 全连接层”, 具体设置上我综合考虑了以下几点:

1. **设计 filter windows:** 设计为 3, 4, 5。
2. **采用 dropout:** 由于数据集量并不大, 所以很容易过拟合, 因此可以采用 dropout 来减缓过拟合。
3. **损失函数:** 由于标签是整数, 很适合用交叉熵损失。
4. **激活函数:** 统一使用 Relu 激活函数。
5. **卷积核数量:** 为了和数据集规模相适应, 我预先使用 50 个卷积核。
6. **学习率:** 考虑到我使用的是 GloVe 预训练词向量, 其一般要求学习率不会很大, 所以预先设置为 0.0005。

相关代码如下:

```
1 # CNN 模型
2 class TextCNN(nn.Module):
3     def __init__(self, vocab_size, embed_dim=EMBED_DIM, num_classes=
4         NUM_CLASSES, filter_sizes=FILTER_SIZES, num_filters=NUM_FILTERS,
5         dropout=DROPOUT, pretrained_embeddings=None):
6         super(TextCNN, self).__init__()
7         self.embedding = nn.Embedding(vocab_size, embed_dim)
8         if pretrained_embeddings is not None:
9             self.embedding.weight.data.copy_(pretrained_embeddings)
10            self.embedding.weight.requires_grad = True # 微调词向量
11        self.convs = nn.ModuleList([
12            nn.Conv2d(1, num_filters, (k, embed_dim)) for k in filter_sizes
13        ])
14        self.dropout = nn.Dropout(dropout)
15        self.fc = nn.Linear(num_filters * len(filter_sizes), num_classes)
```

```
14
15 def forward(self, x):
16     x = self.embedding(x).unsqueeze(1)
17     x = [F.relu(conv(x)).squeeze(3) for conv in self.convs]
18     x = [F.max_pool1d(i, i.size(2)).squeeze(2) for i in x]
19     x = torch.cat(x, 1)
20     x = self.dropout(x)
21     return self.fc(x)
```

2.4 实现卷积神经网络的训练

此部分与正常的神经网络训练并没有什么差别，需要提及的一点设计是：我保存的模型是训练总过程中能得到最高测试集精度的模型，而不是最后一代的模型。这么做的理由是，经过实验，我发现这个数据集上很容易过拟合，训练到后期测试集准确率反而会下降。核心训练部分代码如下：

```
1 for epoch in range(epochs):
2     model.train()
3     total_loss = 0
4     correct, total = 0, 0
5
6     for x, y in train_loader:
7         x, y = x.to(device), y.to(device)
8         optimizer.zero_grad()
9         outputs = model(x)
10        loss = criterion(outputs, y)
11        loss.backward()
12        optimizer.step()
13        total_loss += loss.item()
14        _, predicted = outputs.max(1)
15        total += y.size(0)
16        correct += (predicted == y).sum().item()
17
18    train_acc = correct / total
19    val_acc = evaluate_model(model, val_loader)
20    test_acc = evaluate_model(model, test_loader)
21
22    print(f'Epoch {epoch+1}, Loss: {total_loss/len(train_loader):.4f}, Train
23          Acc: {train_acc:.4f}, Val Acc: {val_acc:.4f}, Test Acc: {test_acc:.4f}'
24          )
```

```
24 # 发现更好的模型时立即保存
25 if test_acc > best_test_acc:
26     print(f'New Best model found, Test Acc: {test_acc:.4f}')
27     best_test_acc = test_acc
28     best_model_state = model.state_dict() # 保存最佳模型状态
29     torch.save(best_model_state, 'best_textcnn_sentiment.pth') # 立即保存最佳模型
```

三、实验

3.1 五分类实验

首先我简要在五分类实验上进行训练和测试，部分运行结果如图 1，最终训练 50 代后测试集精度最高为 45.11%。此外，我绘制了训练过程中训练集、验证集、测试集的准确率的变化图，如图 2。观察这些实验结果，我们能明显发现，尽管训练集精度持续上升，但验证集和测试集精度短暂上升后一直在抖动，因此这个数据集上很可能出现了拟合的问题。据我分析，一方面是由于数据集的量比较小（一万余条），另一方面是电影评论集质量并不高，而且有些词语很难说一定代表证明评价或是负面评价。基于此分析，我尝试了减小学习率，加强正则化、减少卷积核数量等等方法来改善这一点，效果略有提升。

Epoch 1, Loss: 1.5707, Train Acc: 0.2769, Val Acc: 0.3224, Test Acc: 0.3258
New Best model found, Test Acc: 0.3258
Epoch 2, Loss: 1.5233, Train Acc: 0.3302, Val Acc: 0.3451, Test Acc: 0.3430
New Best model found, Test Acc: 0.3430
Epoch 3, Loss: 1.4781, Train Acc: 0.3674, Val Acc: 0.3642, Test Acc: 0.3824
New Best model found, Test Acc: 0.3824
Epoch 4, Loss: 1.4357, Train Acc: 0.3921, Val Acc: 0.3851, Test Acc: 0.3910
New Best model found, Test Acc: 0.3910
Epoch 5, Loss: 1.3914, Train Acc: 0.4208, Val Acc: 0.3878, Test Acc: 0.3982
New Best model found, Test Acc: 0.3982
Epoch 6, Loss: 1.3490, Train Acc: 0.4428, Val Acc: 0.3933, Test Acc: 0.4163
New Best model found, Test Acc: 0.4163
Epoch 7, Loss: 1.3178, Train Acc: 0.4596, Val Acc: 0.3969, Test Acc: 0.4181
New Best model found, Test Acc: 0.4181
Epoch 8, Loss: 1.2835, Train Acc: 0.4750, Val Acc: 0.3960, Test Acc: 0.4186
New Best model found, Test Acc: 0.4186
Epoch 9, Loss: 1.2621, Train Acc: 0.4891, Val Acc: 0.4042, Test Acc: 0.4362
New Best model found, Test Acc: 0.4362
Epoch 10, Loss: 1.2320, Train Acc: 0.5028, Val Acc: 0.4087, Test Acc: 0.4308
Epoch 11, Loss: 1.2089, Train Acc: 0.5146, Val Acc: 0.4096, Test Acc: 0.4452
New Best model found, Test Acc: 0.4452
Epoch 12, Loss: 1.1875, Train Acc: 0.5256, Val Acc: 0.4178, Test Acc: 0.4362

图 1: 训练过程中部分训练集、验证集、测试集的准确率展示。

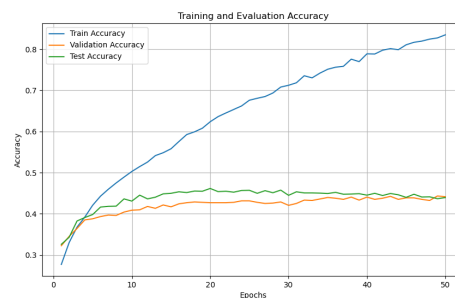


图 2: 训练集、验证集、测试集的准确率随着训练代数的变化图。

3.2 消融实验—超参数变化

为了进一步提高实验精度,同时进行消融实验探索超参数变化的作用,我另写了“grid_search”文件以调用3.1 节中采用的基础代码，采用网格搜索的方式，着重搜索以下参数：

```
1 # 超参数搜索空间
2 param_grid = {
3     'LEARNING_RATE': [1e-4, 5e-4],
4     'FILTER_SIZES': [[3, 4, 5], [3, 5, 7]],
5     'NUM_FILTERS': [50, 128],
6     'DROPOUT': [0.3, 0.5, 0.7],
7     'BATCH_SIZE': [32, 64, 50],
8     'SEED': [42, 1234, 520, 5201314]
9 }
```

据此，我希望能够搜索到更好的一组参数，以提高测试集精度。核心代码如下：

```
1 # 遍历所有超参数组合
2 keys, values = zip(*param_grid.items())
3 for v in itertools.product(*values):
4     params = dict(zip(keys, v))
5     print(f"\n Training with params: {params}")
6 # 设置随机种子
7 set_seed(params['SEED'])
8
9 # (在此后调用基础部分训练代码，传递待试验的参数)
```

运行“grid_search”文件，会得到一系列参数下的准确率和模型，如图 3。最终我得到最好的模型在测试集上达到了 **46.47%** 的准确率，这在视频演示中有展示。我发现，搜索空间中最优的参数组合是学习率为 0.0001, filter_size 为 [3,4,5], num_filters 为 50, drop_out 为 0.5, batch_size 为 50。通过消融实验，我发现超参数还是尽量地以利于防止过拟合为主，比如学习率要小，filter_size 不宜过大，drop_out 率要达标等等。

3.3 拓展实验到二分类任务上

我还利用相同数据集在二分类任务上进行了实验。采取的设计是将原本的 0,1,2,3,4 五个标签合并为(1)“{0,1} + {3,4}”或者(2)“{0} + {4}”两种情形，从而用于二分类。核心代码如下：

```
1 # 修改标签，0 和 1 归为 0，3 和 4 归为 1，滤去标签为 2 的数据
2 self.data = self.data[self.data.iloc[:, 1].isin([0,1,3,4])].reset_index(drop=
    True)
3 self.data.iloc[:, 1] = self.data.iloc[:, 1].apply(lambda x: 0 if x in [0, 1]
    else 1)
```

0.4475_0.0001_50_0.5_64_42.pth
0.4480_0.0001_128_0.5_32_520.pth
0.4493_0.0001_128_0.3_50_42.pth
0.4493_0.0001_128_0.7_32_1234.pth
0.4493_0.0001_50_0.7_32_520.pth
0.4498_0.0001_128_0.3_32_42.pth
0.4502_0.0001_128_0.3_64_520.pth
0.4502_0.0001_128_0.7_50_520.pth
0.4502_0.0001_50_0.5_50_5201314.pth
0.4507_0.0001_128_0.5_50_1234.pth
0.4507_0.0005_50_0.7_64_42.pth
0.4511_0.0001_50_0.5_32_42.pth
0.4516_0.0001_50_0.3_50_5201314.pth
0.4520_0.0001_128_0.3_50_5201314.pth
0.4520_0.0001_128_0.5_64_42.pth
0.4520_0.0005_50_0.3_64_42.pth
0.4520_0.0005_50_0.5_64_5201314.pth
0.4525_0.0001_50_0.3_32_5201314.pth
0.4525_0.0005_50_0.7_32_42.pth

图 3: 网格搜索产生的一系列文件展示。

Epoch 1, Loss: 0.6740, Train Acc: 0.5756, Val Acc: 0.6972, Test Acc: 0.7068
New Best model found, Test Acc: 0.7068
Epoch 2, Loss: 0.6081, Train Acc: 0.6986, Val Acc: 0.7454, Test Acc: 0.7661
New Best model found, Test Acc: 0.7661
Epoch 3, Loss: 0.5494, Train Acc: 0.7552, Val Acc: 0.7661, Test Acc: 0.7781
New Best model found, Test Acc: 0.7781
Epoch 4, Loss: 0.4932, Train Acc: 0.7887, Val Acc: 0.7798, Test Acc: 0.7930
New Best model found, Test Acc: 0.7930
Epoch 5, Loss: 0.4538, Train Acc: 0.8051, Val Acc: 0.7913, Test Acc: 0.8034
New Best model found, Test Acc: 0.8034
Epoch 6, Loss: 0.4213, Train Acc: 0.8185, Val Acc: 0.8039, Test Acc: 0.8138
New Best model found, Test Acc: 0.8138
Epoch 7, Loss: 0.3992, Train Acc: 0.8345, Val Acc: 0.7936, Test Acc: 0.8144
New Best model found, Test Acc: 0.8144
Epoch 8, Loss: 0.3782, Train Acc: 0.8431, Val Acc: 0.7947, Test Acc: 0.8199
New Best model found, Test Acc: 0.8199
Epoch 9, Loss: 0.3628, Train Acc: 0.8510, Val Acc: 0.8016, Test Acc: 0.8177
Epoch 10, Loss: 0.3409, Train Acc: 0.8607, Val Acc: 0.8028, Test Acc: 0.8188
Epoch 11, Loss: 0.3241, Train Acc: 0.8712, Val Acc: 0.8073, Test Acc: 0.8221
New Best model found, Test Acc: 0.8221
Epoch 12, Loss: 0.3148, Train Acc: 0.8769, Val Acc: 0.8062, Test Acc: 0.8210
Epoch 13, Loss: 0.2957, Train Acc: 0.8857, Val Acc: 0.7970, Test Acc: 0.8237
New Best model found, Test Acc: 0.8237
Epoch 14, Loss: 0.2835, Train Acc: 0.8980, Val Acc: 0.8016, Test Acc: 0.8221
Epoch 15, Loss: 0.2692, Train Acc: 0.9010, Val Acc: 0.8062, Test Acc: 0.8248
New Best model found, Test Acc: 0.8248
Epoch 16, Loss: 0.2611, Train Acc: 0.9071, Val Acc: 0.8062, Test Acc: 0.8248
Epoch 17, Loss: 0.2473, Train Acc: 0.9134, Val Acc: 0.8062, Test Acc: 0.8276
New Best model found, Test Acc: 0.8276
Epoch 18, Loss: 0.2387, Train Acc: 0.9146, Val Acc: 0.8062, Test Acc: 0.8254
Epoch 19, Loss: 0.2283, Train Acc: 0.9223, Val Acc: 0.8073, Test Acc: 0.8259
Epoch 20, Loss: 0.2140, Train Acc: 0.9327, Val Acc: 0.8073, Test Acc: 0.8276

图 4: 训练过程中部分训练集、验证集、测试集的准确率展示。

Epoch 1, Loss: 0.6810, Train Acc: 0.5534, Val Acc: 0.5888, Test Acc: 0.6283
New Best model found, Test Acc: 0.6283
Epoch 2, Loss: 0.6333, Train Acc: 0.6584, Val Acc: 0.8224, Test Acc: 0.7861
New Best model found, Test Acc: 0.7861
Epoch 3, Loss: 0.5799, Train Acc: 0.7500, Val Acc: 0.8289, Test Acc: 0.8038
New Best model found, Test Acc: 0.8038
Epoch 4, Loss: 0.5379, Train Acc: 0.7870, Val Acc: 0.8092, Test Acc: 0.8024
Epoch 5, Loss: 0.4818, Train Acc: 0.8282, Val Acc: 0.8717, Test Acc: 0.8378
New Best model found, Test Acc: 0.8378
Epoch 6, Loss: 0.4387, Train Acc: 0.8546, Val Acc: 0.8783, Test Acc: 0.8451
New Best model found, Test Acc: 0.8451
Epoch 7, Loss: 0.4040, Train Acc: 0.8693, Val Acc: 0.8750, Test Acc: 0.8687
New Best model found, Test Acc: 0.8687
Epoch 8, Loss: 0.3713, Train Acc: 0.8660, Val Acc: 0.9013, Test Acc: 0.8746
New Best model found, Test Acc: 0.8746
Epoch 9, Loss: 0.3340, Train Acc: 0.8941, Val Acc: 0.9013, Test Acc: 0.8717
Epoch 10, Loss: 0.3140, Train Acc: 0.8979, Val Acc: 0.9112, Test Acc: 0.8835
New Best model found, Test Acc: 0.8835
Epoch 11, Loss: 0.2885, Train Acc: 0.9038, Val Acc: 0.9046, Test Acc: 0.8820
Epoch 12, Loss: 0.2670, Train Acc: 0.9118, Val Acc: 0.9079, Test Acc: 0.8864
New Best model found, Test Acc: 0.8864
Epoch 13, Loss: 0.2545, Train Acc: 0.9197, Val Acc: 0.9079, Test Acc: 0.8835
Epoch 14, Loss: 0.2367, Train Acc: 0.9269, Val Acc: 0.9013, Test Acc: 0.8850
Epoch 15, Loss: 0.2200, Train Acc: 0.9307, Val Acc: 0.9112, Test Acc: 0.8835
Epoch 16, Loss: 0.2086, Train Acc: 0.9378, Val Acc: 0.9013, Test Acc: 0.8835
Epoch 17, Loss: 0.1981, Train Acc: 0.9424, Val Acc: 0.9046, Test Acc: 0.8894
New Best model found, Test Acc: 0.8894
Epoch 18, Loss: 0.1895, Train Acc: 0.9412, Val Acc: 0.9079, Test Acc: 0.8879
Epoch 19, Loss: 0.1789, Train Acc: 0.9529, Val Acc: 0.9079, Test Acc: 0.8850
Epoch 20, Loss: 0.1687, Train Acc: 0.9521, Val Acc: 0.9079, Test Acc: 0.8835

图 5: 训练集、验证集、测试集的准确率随着训练代数的变化图。

(1)和(2)两种情形下的二分类前二十代的准确率分别如图 4和图 5。
充分训练 50 代后,(1)和(2)两种情形下能达到的最高测试集精度分别为 **83.36%** 和 **89.53%**。
分析上述结果,我们能够发现,二分类准确率远远高于五分类准确率,这是非常合理的,因为这里二分类相比五分类的唯一区别是我们将标签做了简化,二分类的难度是远远低于五分类的。再进一步,(1)情形相比(2)情形,准确率会低一点,因为(2)情形考虑的情形更加

简单，只有原本标签为 0 和 4，对应电影评价情绪为“非常差”和“非常好”这两种极端。据此，我们可以发现，数据集的设置和处理往往会影响着实验的最终效果。

3.4 与往年参考准确率对比

由于我参考的论文并没有在 Rotten Tomatoes 数据集上实验，所以我请求老师给了往届的该项目参考准确率，往届准确率如图 6所示。可以发现，我的实验准确率和往届完全相当。

实验结果	
5cls_acc	2cls_acc
0.4624	0.8515

```
Batch[8500/8544] - loss: 0.591844 acc: 80.0000%(40/50)
Dev Accuracy: Evaluation - loss: 0.000160 acc: 43.9600%(404/1101)
Test Accuracy: Evaluation - loss: 0.000469 acc: 46.3801%(1025/2210)
The Current Best Dev Accuracy: 45.4133, and Test Accuracy is :46.2443, locate on 32 epoch.

Batch[10/6531] - loss: 0.103791 acc: 98.0000%(49/50)
Batch[20/6531] - loss: 0.096644 acc: 96.0000%(48/50)
Batch[30/6531] - loss: 0.069270 acc: 100.0000%(50/50)
Batch[40/6531] - loss: 0.099424 acc: 98.0000%(49/50)
Batch[50/6531] - loss: 0.083702 acc: 98.0000%(49/50)
Batch[60/6531] - loss: 0.056586 acc: 100.0000%(50/50)
Batch[70/6531] - loss: 0.195197 acc: 94.0000%(47/50)
Batch[80/6531] - loss: 0.088209 acc: 100.0000%(50/50)
Batch[90/6531] - loss: 0.083358 acc: 98.0000%(49/50)
Batch[100/6531] - loss: 0.068495 acc: 98.0000%(49/50)
Dev Accuracy: Evaluation - loss: 0.000516 acc: 82.9476%(681/821)
Test Accuracy: Evaluation - loss: 0.000242 acc: 84.1197%(1462/1738)
The Current Best Dev Accuracy: 83.1912, and Test Accuracy is :85.1554, locate on 39 epoch.
```

图 6: 往届该项目准确率

3.5 案例分析

我通过编写代码，分析了五分类任务上训练完成后的一些正确预测的样例和错误预测的样例，它们本身的标签都是 0，也就是态度为非常讨厌。分别如图 7和图 8所示。图 7中我们能发现红色框中的词语都明显包含着非常负面的情绪，因此模型能正确预测是合理的。而图 8中我认为是模型在认知中出现了偏差，尤其是红色框圈起来的：模型可能认为“plausible”是一个证明的词，但是忽略了前面的 less；同时，模型可能认为“aptly enough”也是一个正面的词语，却忽略了后面跟的是“punishment”。通过案例分析，我们也能理解这个任务的艰难性，尽管已经有了卷积神经网络捕捉相邻词语间的关系，但语言方面的理解还是很难做到很好。

四、总结与反思

经过本次实验，我有一些总结和反思如下：

=== Correctly Predicted Examples ===

Label 0 - Correct Predictions:

1. Sentence: after that , it just gets **stupid** and **maudlin**
True: 0, Predicted: 0
2. Sentence: a sermonizing and **lifeless** paeen to teenage **dullards**
True: 0, Predicted: 0
3. Sentence: a **dreadful** live action movie
True: 0, Predicted: 0
4. Sentence: a **boring** , formulaic mix of serial **killers** and stalk 'n' slash
True: 0, Predicted: 0
5. Sentence: it 's not **horrible** , just **horribly mediocre**
True: 0, Predicted: 0

图 7: 正例

=== Misclassified Examples ===

True Label 0 - Wrong Predictions:

1. Sentence: the story loses its bite in a last minute happy ending that 's even **less plausible** than the rest of the picture
True: 0, Predicted: 1
2. Sentence: in a normal screen process , these bromides would be barely enough to sustain an interstitial program on the discovery channel
True: 0, Predicted: 1
3. Sentence: watching haneke 's film is , aptly **enough** , a challenge and a punishment
True: 0, Predicted: 3
4. Sentence: her film is unrelentingly claustrophobic and unpleasant
True: 0, Predicted: 1
5. Sentence: a depressing confirmation of everything those of us who do n't object to the description `` unelected '' have suspected all along george w bush i s an incurious , uncharismatic , overgrown frat boy with a mean streak a mile wide
True: 0, Predicted: 1

图 8: 负例

- **数据集很大程度上决定了训练质量：**一开始我发现我的五分类测试集准确率只有四十几的时候，非常惊讶，以为是自己代码写错了。我直接询问了老师应该达到的参考准确率，虽说解决了这个问题，但是还是欠缺一些思考。考虑到数据集的大小，它只包含一万个样本左右，并且样本质量不高，数据集含有不少语义信息，且这些电影评价中有些词很难区别正面还是负面，所以最终精度不高是应该能推理出来的。
- **如何应对容易过拟合的数据集：**首先是需要发现过拟合，我通过绘制图 2, 直观观察训练集、验证集、测试集精度随着训练代数的变化，发现了尽管训练集精度持续上升，但验证集和测试集精度短暂上升后一直在抖动，断定出现了过拟合。为了减缓这一点，我综合采用了减小学习率、减少卷积核数量、增加 dropout 的方法，取得了更好的效果。
- **采用网格搜索提高精度：**网格搜索在低复杂度情况下是个很好地优化模型的方法，鉴于这个实验本身不是特别耗时，我考虑使用网格搜索的方法寻找更好的参数，提高精度高于 1%。
- **比较只有在公平的情况下才有意义：**观察图 1, 图 4和图 5, 我们发现仅仅修改任务类别的数量、标签的划分方式就能大大影响实验的结果，因此这也提醒我，在做卷积神经网络效果比较或者是其它机器学习算法、模型比较的时候，一定要注重公平，尽量控制无关变量一定，从而凸显效果。

参考文献

- [1] Yoon Kim. Convolutional neural networks for sentence classification. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, pages 1746–1751, 2014.
- [2] Jeffrey Pennington, Richard Socher, and Christopher D Manning. Glove: Global vectors for word representation. In *Proceedings of the 2014 conference on empirical methods in natural language processing*, pages 1532–1543, 2014.